```
import pandas as pd
import numpy as np
```

```
import pandas as pd
import numpy as np

df = pd.read_csv("/content/retail_dataset.csv")

df.head()
```

	Transaction ID	Customer ID	Product Category	Quantity	Price	Date	Region
0	T001	C015	Books	7.0	167.0	02-02-2023	North
1	T002	C004	Toys	4.0	374.0	17-02-2023	South
2	T003	C033	Furniture	8.0	1059.0	17-05-2024	South
3	T004	C047	Groceries	5.0	304.0	11-02-2024	South
4	T005	C022	Books	7.0	282.0	27-07-2024	West

Next steps: Generate code with df New interactive sheet

Question 1: Load the dataset and display basic information

Explanation: In this question, the dataset is loaded using the pandas library. The head() function is used to display the first five rows of the dataset, and the info() function provides details about column names, data types, and missing values.

```
import pandas as pd
import numpy as np

df = pd.read_csv("/content/retail_dataset.csv")

df.head()
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 7 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Transaction ID    100 non-null    object
 1   Customer ID       100 non-null    object
 2   Product Category  97 non-null     object
 3   Quantity          98 non-null     float64
 4   Price             95 non-null     float64
 5   Date              100 non-null    object
 6   Region            98 non-null     object
dtypes: float64(2), object(5)
memory usage: 5.6+ KB
```

Question 2: Inspect the dataset and identify missing values

Explanation: This step is used to check how many missing (null) values are present in each column of the dataset. It helps in understanding data quality.

```
df.isnull().sum()
```

	0
Transaction ID	0
Customer ID	0
Product Category	3
Quantity	2
Price	5
Date	0
Region	2

dtype: int64

Question 3: Replace missing values in Quantity and Price

Explanation: Missing values in the Quantity and Price columns are replaced with their respective mean values. This ensures that the dataset remains consistent.

```
df['Quantity'].fillna(df['Quantity'].mean(), inplace=True)
df['Price'].fillna(df['Price'].mean(), inplace=True)
```

```
/tmp/ipykernel_20254/3913199803.py:1: FutureWarning: A value is trying to be set on a copy of a DataFram
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate obj

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplac


  df['Quantity'].fillna(df['Quantity'].mean(), inplace=True)
/tmp/ipykernel_20254/3913199803.py:2: FutureWarning: A value is trying to be set on a copy of a DataFram
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate obj

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplac


  df['Price'].fillna(df['Price'].mean(), inplace=True)
```

Question 4: Remove rows with missing values

Explanation: Rows that contain missing values in Product Category or Region columns are removed to improve data accuracy.

```
df.dropna(subset=['Product Category', 'Region'], inplace=True)
```

Question 5: Create a new column Revenue

Explanation: A new column called Revenue is created by multiplying Quantity and Price. It represents total earnings from each transaction.

```
df['Revenue'] = df['Quantity'] * df['Price']
df.head()
```

	Transaction ID	Customer ID	Product Category	Quantity	Price	Date	Region	Revenue
0	T001	C015	Books	7.0	167.0	02-02-2023	North	1169.0
1	T002	C004	Toys	4.0	374.0	17-02-2023	South	1496.0
2	T003	C033	Furniture	8.0	1059.0	17-05-2024	South	8472.0
3	T004	C047	Groceries	5.0	304.0	11-02-2024	South	1520.0
4	T005	C022	Books	7.0	282.0	27-07-2024	West	1974.0

Next steps: Generate code with df New interactive sheet

Question 6: Calculate total revenue

Explanation: NumPy is used to calculate the total revenue by summing all values in the Revenue column.

```
import numpy as np

total_revenue = np.sum(df['Revenue'])
print("Total Revenue:", total_revenue)
```

```
Total Revenue: 465529.66058002145
```

Question 7: Revenue by product category

Explanation: The dataset is grouped by Product Category, and the total revenue for each category is calculated.

```
category_revenue = df.groupby('Product Category')['Revenue'].sum()
print(category_revenue)
```

```
Product Category
Beauty          27647.000000
Books           29597.938776
Clothing        85006.000000
Electronics     93204.315789
Furniture      129664.526316
Groceries       11978.000000
Sports          34484.000000
Toys            53947.879699
Name: Revenue, dtype: float64
```

Question 8: Top and Bottom categories

Explanation: The top 3 categories with highest revenue and bottom 3 categories with lowest revenue are identified.

```
top3 = category_revenue.sort_values(ascending=False).head(3)
bottom3 = category_revenue.sort_values().head(3)

print("Top 3 Categories:\n", top3)
print("Bottom 3 Categories:\n", bottom3)
```

```
Top 3 Categories:
 Product Category
Furniture      129664.526316
Electronics     93204.315789
Clothing        85006.000000
Name: Revenue, dtype: float64
Bottom 3 Categories:
 Product Category
Groceries    11978.000000
```

```
Beauty       27647.000000
Books        29597.938776
Name: Revenue, dtype: float64
```

Question 9: Revenue by region

Explanation: The dataset is grouped by Region to calculate total revenue in each region. The highest and lowest revenue regions are identified.

```
region_revenue = df.groupby('Region')['Revenue'].sum()

print(region_revenue)
print("Highest Revenue Region:", region_revenue.idxmax())
print("Lowest Revenue Region:", region_revenue.idxmin())
```

```
Region
East     126232.044039
North    153244.353383
South    104855.736842
West      81197.526316
Name: Revenue, dtype: float64
Highest Revenue Region: North
Lowest Revenue Region: West
```

Question 10: Monthly revenue and statistics

Explanation: The Date column is converted into datetime format to extract the month. Monthly revenue is calculated. Additionally, mean, median, and standard deviation of Revenue are computed using NumPy.

```
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True, errors='coerce')

df['Month'] = df['Date'].dt.month

monthly_revenue = df.groupby('Month')['Revenue'].sum()

print(monthly_revenue)
```

```
Month
1     43994.000000
2     33028.000000
3     10557.000000
4     57298.526316
5     27915.000000
6     28190.000000
7     47851.292159
8     76958.000000
9     30719.105263
10    38017.000000
11    27662.736842
12    43339.000000
Name: Revenue, dtype: float64
```

Start coding or generate with AI.